

Modul: Data & Process Integration
Studiengang: Strategisches Informationsmanagement
Semester: SoSe2026
Dozent: Prof. Dr. Markus Grüne

Fallstudie Tierarztpraxis Canini - Dokumentation -



Name: Jenisa Sivagnanalingham (1121105)
Team: zorro (Szenario B)

Abgabedatum: 5. Juli 2026

1. Projektaufgabe

Gegenstand dieser Fallstudie ist der Zusammenschluss von Patientendaten aus vier Tierarztpraxen (Juckstadt, Waldrand, Schmidt und Bergblick) in einer zentralen Datenbank. Die Kernherausforderung liegt in der Harmonisierung der heterogenen Rohdaten, da die Datensätze in unterschiedlichen Formaten (CSV, JSON, XML) vorliegen und qualitative Mängel aufweisen. Ziel ist es, diese Daten zu bereinigen und in ein einheitliches Zielschema (Golden Records) zu überführen.

Im Rahmen des spezifischen Szenarios B wird für die Dublettenerkennung ein KI-gestützter Ansatz verfolgt. Anstelle fehleranfälliger, starrer SQL-Regeln (z. B. Regex) identifiziert eine Kombination aus semantischer Vektorsuche und einem lokalen Large Language Model (LLM) redundante Datensätze.

2. Data-Profiling

Zunächst sollten die Rohdaten untersucht werden, um die vorliegende Datenqualität zu bewerten und ein Mapping für das spätere Zielschema vorzubereiten. Hierbei wurden signifikante Probleme identifiziert, die eine Tabellenzusammenführung erschweren, wie z. B.:

- **Fehlende referentielle Integrität:** In den Praxen Juckstadt und Schmidt fehlen eindeutige Kunden-IDs (Foreign Keys) in den Behandlungsdaten. Die Verknüpfung erfolgt unsauber über Textfelder (Nachname + Initiale).
- **Schema-Unterschiede:** Bergblick kombiniert Vor- und Nachnamen, während andere Praxen diese trennen. Waldrand verwendet rein englische Attribute (z. B. `customer_id`, `species`).
- **Semantische Fehler & Tippfehler:** Hohe Anzahl an Buchstabendrehern (z. B. „Pteers“, „Wlof“) sowie Vornamen, die nur als Initialen vorliegen.
- **Format-Inkonsistenzen:** Vermischung von US-Datumsformaten (MM/DD/YYYY), deutschen Formaten und ISO 8601, sowie inkonsistente Währungsformate (Komma vs. Punkt).

Diese Defizite sind im „Data Dictionary“ dokumentiert. Dieses dient als Grundlage für die spätere Datenbereinigung und stellt sicher, dass alle Attribute trotzdem korrekt in das einheitliche Zielschema überführt werden können.

3. Staging

Der Integrationsprozess begann mit dem Staging der Quelldaten. Insgesamt wurden 916 Kunden- und 600 Behandlungsdatensätze aus den vier Praxissystemen in die Staging-Area einer DuckDB-Datenbank geladen.

Der Import wurde je nach Dateiformat spezifisch gelöst:

- **CSV (Juckstadt, Waldrand, Schmidt):** Eingelesen über pandas, unter Berücksichtigung unterschiedlicher Trennzeichen (Komma vs. Semikolon).
- **JSON (Schmidt Behandlungen):** Hierarchische Strukturen wurden abgeflacht („Flattening“), um eine zweidimensionale Tabellenstruktur zu erhalten.
- **XML (Bergblick):** Das Modul xml.etree.ElementTree wurde genutzt, um die Knoten auszulesen und tabellarisch zu mappen.

Zur lückenlosen Rückverfolgbarkeit erhielt jeder Datensatz über die Funktion `row_number()` eine eindeutige `quell_zeile`.

Bereits in dieser Phase zeigte das Data Profiling massive strukturelle und inhaltliche Diskrepanzen:

- **Identifikatoren:** Die Praxen nutzten grundverschiedene ID-Konzepte (z. B. einfache Integer 1 vs. Strings P-4001).
- **Namenskonventionen:** Praxis 1 und 2 trennten Vor- und Nachnamen, während Praxis 3 die Namen kombinierte und Vornamen oft nur als Initialen angab (z. B. Schneider X.).
- **Fremdschlüssel-Problematik:** Eine zentrale Hürde war die Referenzierung der Behandlungen. Einige Praxen (Juckstadt, Schmidt) nutzten anstelle von numerischen IDs den Nachnamen als Fremdschlüssel, was relationale Standard-Joins erschwert.

Das Staging-Modul überführte diese Daten unverändert, aber strukturiert in initiale Tabellen, um die Nachvollziehbarkeit sicherzustellen.

4. Transformation

Um die Daten für die KI-Verarbeitung vorzubereiten, musste eine Transformation implementiert werden. In der ersten Iteration des Projekts, wurde diese nicht richtig ausgeführt und dadurch zeigten sich hier erhebliche Stabilitätsprobleme, die durch tiefgreifende Architekturadaptierungen gelöst wurden.

Architektur-Fix: Globale Kunden-ID

Ursprünglich wurden Praxis-ID und Quell-ID zu Text-Strings kombiniert (z. B. 1_P001). Dies führte in den späteren Matching- und Join-Prozessen zu Performance-Einbußen und Typisierungsfehlern. Als Lösung wurde in der Transform-Schicht eine saubere, fortlaufende globale `kunde_id` (BIGINT) eingeführt.

Zusätzlich wurden die Kontaktdaten normalisiert: Telefonnummern wurden in das internationale E.164-Format überführt, Datumsformate standardisiert und NULL-Werte vereinheitlicht. Das Resultat war die Tabelle `transform.norm_kunde` mit 916 harmonisierten, aber noch redundanten Datensätzen.

Die Problematik in der Behandlungstabelle, mit den uneindeutigen Fremdschlüssel der Patienten wurde im Verlauf des Projektes umgangen, aber nicht gelöst. Für die Duplikats-Erkennung macht das keinen Unterschied, aber spätestens für Szenario C müsste hier eine Lösung gefunden werden.

5. Embedding & Vector Search

Da ein direkter Kreuzvergleich aller 916 Patienten durch ein LLM zu über 419.000 Anfragen geführt hätte, wurde der Suchraum in zwei Schritten effizient reduziert. Alle KI-Modelle liefen aus Datenschutzgründen lokal über das Ollama-Framework.

Embedding

Um Tippfehler und abweichende Strukturen zu umgehen, folgte die semantische Vektorisierung (02_embeddings.py). Die Kundendaten wurden zu Text-Strings (Name | Adresse | PLZ/Ort | Telefon) zusammengefasst und durch das Modell nomic-embed-text in 768-dimensionale Vektoren übersetzt.

Das Modell erfasst dadurch die inhaltliche Bedeutung. Ähnliche Patienten werden nah beieinander in einem mathematischen Raum angeordnet, sodass Dubletten trotz starker Schreibfehler zuverlässig identifiziert werden können.

Vector Search:

Auf Basis der berechneten Vektoren wurde ein Hierarchical Navigable Small World (HNSW) Index in DuckDB erstellt (03_vector_search.py). Über eine K-Nearest-Neighbor (KNN) Suche wurden für jeden Kunden die 10 ähnlichsten Nachbarn anhand der Kosinus-Distanz identifiziert. Dieser Schritt reduzierte den Suchraum erfolgreich auf 2.406 hochgradig verdächtige Kandidatenpaare.

6. LLM-Judge

Die 2.406 Kandidatenpaare bildeten die Arbeitsgrundlage für die eigentliche Dublettenprüfung. Als "Judge" kam das Modell qwen2.5:7b-instruct zum Einsatz, dass die Paare qualitativ bewertet (Prüfung auf Tippfehler, Initialen, Adressdreher, etc.).

Architektur-Fix: Ausfallsichere JSON-Validierung

In frühen Projektphasen brachen Skripte ab, wenn das LLM unsauber formatierte Antworten lieferte. Zur Stabilisierung wurde das Pydantic-Framework integriert. Das LLM wurde gezwungen, strikt nach einem definierten Schema (MatchEntscheidung) zu antworten. Fehlerhafte Antworten triggern nun eine Retry-Logik; schlägt diese fehl, wird das Paar mit einem SKIP übersprungen, anstatt die Datenbank zu korrumpieren. Dieser Fix machte die Pipeline ausfallsicher. Das Modell lieferte neben der binären Entscheidung eine Confidence-Metrik und eine textuelle Begründung (Reasoning) für jede Entscheidung. Zudem wurde der Prompt an das LLM nun 1-zu-1 aus dem Beispiel-Code übernommen, da vorher Telefonnummern und E-Mails nicht so streng gewertet wurden, und dadurch viele Dubletten „identifiziert“ wurden, die eigentlich keine waren.

Aufgrund der Performance wurden nur die Top 30 Paare überprüft. Auffällig war zudem, dass Paare mit einem Konfidenzwert von unter 1 immer als 2-stellige Zahl (bspw. 95) berechnet und dadurch übersprungen wurden. Möglicherweise müsste der Prompt angepasst werden.

7. Clustering

Nachdem das LLM einzelne Dubletten-Paare ($A=B$) verifiziert hatte, mussten diese zu logischen Einheiten zusammengefasst werden. Hierzu wurde graphentheoretisch die transitive Hülle berechnet, um Dubletten-Ketten ($A=B$ und $B=C$ bedeutet $A=B=C$) aufzulösen.

Architektur-Fix: Zuweisung von Single-Kunden

Ein massives Problem früherer Iterationen bestand darin, dass Kunden ohne jegliche Dubletten (Singletons) aufgrund leerer Spalten in ein einziges, fehlerhaftes Riesen-Cluster gemergt wurden. Dies wurde gelöst, indem DuckDB-spezifische Fensterfunktionen (`ROW_NUMBER()`) und der Befehl `EXCLUDE` genutzt wurden. Jeder eindeutige Kunde erhielt so garantiert eine exklusive, dedizierte `cluster_id`. Aus den identifizierten Dubletten entstanden 22 saubere Cluster.

8. Golden Records

Der finale Integrationsschritt bestand in der Beladung des Zielschemas durch die Erzeugung sogenannter "Golden Records" (die "Single Source of Truth"). Hierbei wurden die Cluster zu jeweils einem sauberen Datensatz zusammengeführt, wodurch die ursprüngliche Menge von 916 Kunden auf 893 bereinigte Golden Records reduziert wurde.

Architektur-Fix: Verhinderung von Frankenstein-IDs

Beim anfänglichen Verschmelzen (Merging) der Dubletten kam es vor, dass Attribute aus verschiedenen Rohdatensätzen wild gekreuzt wurden, was zu Datenbank-Fehlern (Constraint Violations) führte. Durch die Implementierung der Funktion `arg_max()` wurde sichergestellt, dass die Master-Attribute für den Golden Record immer aus exakt demselben verifizierten Ursprungsdatensatz extrahiert werden.

Architektur-Fix: Rettung verlorener Behandlungen

Aufgrund der in Phase 1 identifizierten "Dirty Data" (Nachnamen als Fremdschlüssel bei Juckstadt und Schmidt) gingen beim initialen Zusammenführen 150 Behandlungsdatensätze verloren. Die Mapping-Logik wurde grundlegend überarbeitet: Durch den Einsatz eines robusten `LEFT JOIN`, der flexibel entweder auf die ID oder den Nachnamen (case-insensitive) prüft, konnten sämtliche 600 Behandlungen verlustfrei auf die neuen Golden-Record-IDs umgebogen und im Zielschema (`verbund_behandlung`) gespeichert werden. Um zudem saubere und vorzeigbare

Primärschlüssel (startend bei 1) für das Endprodukt zu gewährleisten, wurde eine DROP SEQUENCE-Logik vor der finalen Beladung implementiert.

7. Evaluierung gegen Goldstandard

Zur Validierung wurde die Leistung des LLM-Judges gegen einen manuell erstellten Goldstandard getestet. Wegen der begrenzten Hardware-Ressourcen wurde der LLM-Testlauf auf 30 Kandidatenpaare limitiert.

Metriken der Evaluierung:

Evaluierungsergebnisse:

- Gefundene Dubletten-Paare (Modell): 24
- Echte Dubletten-Paare (Goldstandard): 144
- Korrekt gefunden (True Positives): 24
- Falsch verknüpft (False Positives): 0
- Übersehen (False Negatives): 120

Berechnete Metriken:

- **Precision (Genauigkeit): 100.00 %**
- **Recall (Trefferquote): 16.67 %**
- **F1-Score: 28.57 %**

Interpretation:

Der F1-Score von rund 29 % und der Recall von knapp 17 % sind direkte Artefakte der künstlichen Limitierung auf 30 Aufrufe und spiegeln nicht das Limit der KI wider. Von zentraler Bedeutung ist die **Precision von 100 %**. Das System hat im gesamten Testlauf keine einzige falsche Verknüpfung vorgenommen (0 False Positives). Im Kontext medizinischer Akten, wo die fehlerhafte Zusammenlegung zweier verschiedener Patienten fatale Konsequenzen haben kann, ist dies die kritischste Metrik.

8. Fazit und Reflexion

Das Projekt demonstriert, dass eine lokale, KI-gestützte Data-Engineering-Pipeline im Gegensatz zu herkömmlichen SQL-Skripten die Verarbeitung stark inkonsistenter Datensätze deutlich vereinfacht. Durch die Kombination aus intelligenter Vector-Search-Filterung und lokalen Sprachmodellen als Entscheidungsinstanz beweist das System ein tiefes logisches Kontextverständnis.

Obwohl die rechenintensive lokale Vektorisierung und der sogenannte LLM-Judge anfangs zu langen Ausführungszeiten führten, konnte das System durch iterative Architekturoptimierungen entscheidend stabilisiert werden. Maßnahmen wie striktes Pydantic-Parsing, dynamisches Clustering und tolerante Joins verwandelten den initialen Prototypen in eine ausfallsichere Lösung, die fehlerhafte Daten zuverlässig in ein sauberes relationales Schema überführt.

Es gilt jedoch zu beachten, dass der gesamte Prozess äußerst rechenintensiv ist. Daher wurden im initialen Lauf des LLM-Judges lediglich die Top 30 von insgesamt 2.600 Datenpaaren analysiert. Von diesen mussten zudem sieben Paare übersprungen werden, da das Modell die Konfidenzwerte fehlerhaft als Prozentzahlen ausgab. Dies geschah, obwohl sowohl der Prompt als auch die Pydantic-Anweisung strikt vorgaben, die Werte im Intervall zwischen 0 und 1 zu halten. Hier ist voraussichtlich eine weitere Präzisierung des Prompts erforderlich.

Um Leistungsmetriken wie F1-Score und Recall valide evaluieren und die daraus resultierende Precision ermitteln zu können, sind zusätzliche Iterationen mit einer größeren Stichprobe und Anpassungen unumgänglich, um eine konkrete Aussage zu treffen, ob die LLM der Standard SQL-Methodik wirklich überlegen ist.